

Warm up

A `struct` `Mystery` is defined as follows:

```
struct Mystery {  
    int *a;  
    int *b;  
};
```

Assume `m` is a pointer to a `struct` `Mystery` object. The value of the pointer is in register `x0`. What assembly code would you use to place the (a) *address* of `m->b` into `x1`, and (b) *value* of `m->b` into `x2`?

- (a) `add x1, x0, 8`
 - `m->b` is located at an offset of 8 bytes from the start of the struct, since `m->a` is at offset 0 and is 8 bytes long
- (b) `ldr x2, [x0, 8]` (alternatively: `ldr x2, [x1]`)

Precept 20: Assignment 5 and BigInt_add

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

April 14, 2026

Overview

- 1 BigInt_add
- 2 Your turn: bug hunting
- 3 Summary

BigInt and BigInt_add

```
enum {MAX_DIGITS = 32768}; /* arbitrary */

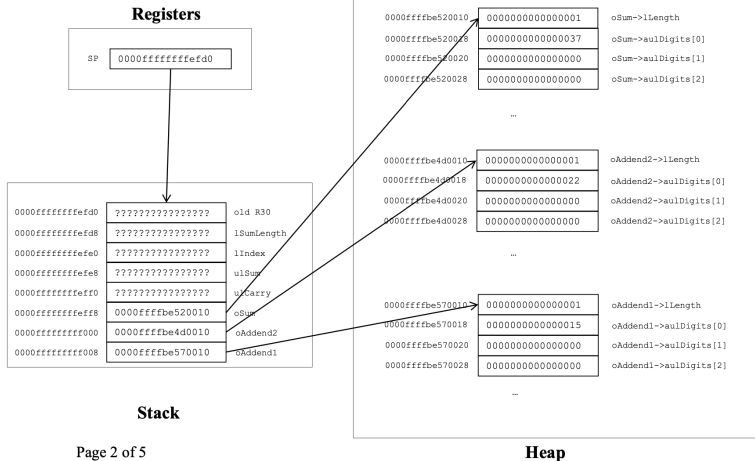
struct BigInt {
    long lLength;
    unsigned long aulDigits[MAX_DIGITS];
};

int BigInt_add(BigInt_T oAddend1, BigInt_T oAddend2,
               BigInt_T oSum) {
    unsigned long ulCarry;
    unsigned long ulSum;
    long lIndex;
    long lSumLength;
    ...
}
```

BigInt_add: normal pattern

Princeton University
COS 217: Introduction to Programming Systems
The BigInt_add Function: Memory Map: Normal Pattern

Your addresses may differ



How to access oAddend2->aulDigits[2] in assembly?

Using register addressing:

```
mov x0, sp          // x0 has 0000fffffffffed0 (hex)
                    // x0 has the addr of the top of stack
add x0, x0, 48       // x0 has 0000fffffffff000
                    // x0 has &oAddend2
ldr x0, [x0]         // x0 has 0000ffffbe4d0010 (hex)
                    // x0 has oAddend2
add x0, x0, 8        // x0 has 0000ffffbe4d0018(hex)
                    // x0 has oAddend2->aulDigits
mov x1, 2            // x1 has 0000000000000002(hex)
                    // x1 has the index
lsl x1, x1, 3         // x1 has 0000000000000010(hex)
                    // x1 has a byte offset
add x0, x0, x1       // x0 has 0000ffffbe4d0028(hex)
                    // x0 has oAddend2->aulDigits + 2
ldr x0, [x0]         // x0 has 0000000000000000(hex)
                    // x0 has *(oAddend2->aulDigits + 2)
                    // x0 has oAddend2->aulDigits[2]
```

How to access oAddend2->aulDigits[2] in assembly?

Using scaled register offset addressing:

```
ldr x0, [sp, 48]           // x0 has 0000ffffbe4d0010(hex)
                           // x0 has oAddend2
add x0, x0, 8              // x0 has 0000ffffbe4d0018(hex)
                           // x0 has oAddend2->aulDigits
mov x1, 2                  // x1 has 0000000000000002(hex)
                           // x1 has the index
ldr x0, [x0, x1, lsl 3]    // x0 has 0000000000000000(hex)
                           // x0 has oAddend2->aulDigits[2]
```

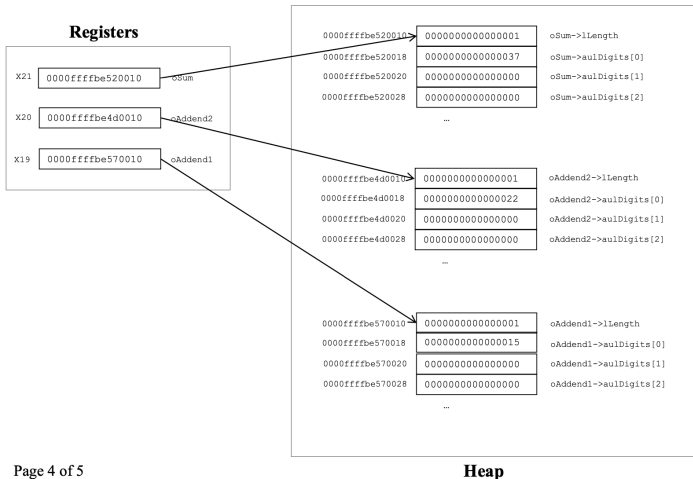
BigInt_add: optimised pattern

Princeton University

COS 217: Introduction to Programming Systems

The BigInt_add Function: Memory Map: Optimized Pattern

Your addresses
may differ



Optimised: accessing oAddend2->aulDigits[2]

Using register addressing:

```
mov x0, x20      // x0 has 0000ffffbe4d0010 (hex)
                  // x0 has oAddend2
add x0, x0, 8     // x0 has 0000ffffbe4d0018(hex)
                  // x0 has oAddend2->aulDigits
mov x1, 2         // x1 has 0000000000000002(hex)
                  // x1 has the index
lsl x1, x1, 3     // x1 has 0000000000000010(hex)
                  // x1 has a byte offset
add x0, x0, x1    // x0 has 0000ffffbe4d0028(hex)
                  // x0 has oAddend2->aulDigits + 2
ldr x0, [x0]      // x0 has 0000000000000000(hex)
                  // x0 has *(oAddend2->aulDigits + 2)
                  // x0 has oAddend2->aulDigits[2]
```

Optimised: accessing oAddend2->aulDigits[2]

Using scaled register offset addressing:

```
mov x0, x20          // x0 has 0000ffffbe4d0010 (hex)
                     // x0 has oAddend2
add x0, x0, 8         // x0 has 0000ffffbe4d0018(hex)
                     // x0 has oAddend2->aulDigits
mov x1, 2             // x1 has 0000000000000002(hex)
                     // x1 has the index
ldr x0, [x0, x1, lsl 3] // x0 has 0000000000000000(hex)
                     // x0 has *(oAddend2->aulDigits + 2)
                     // x0 has oAddend2->aulDigits[2]
```

A note on using gdb

- Can use gdb on BigInt_add when debugging flat code or when debugging assembly code
- gdb allows you to:
 - set conditional breakpoints (e.g., b foo.c:10 if x0 == 0)
 - look at register contents (e.g., info registers or p \$x0)
 - examine memory (e.g., x \$sp+8)
- Use simple.c or fib.c client code to debug bigintadd.s or bigintaddflat.c
 - gcc217 -g simple.c bigint.c bigintadd.s -o simples
 - gcc217 -g fib.c bigint.c bigintadd.s -o fibs
 - gcc217 -g fib.c bigint.c bigintaddflat.c -o fibc

Overview

1 BigInt_add

2 Your turn: bug hunting

3 Summary

Bug hunting

Identify and fix the bug in the following snippets:

- Load oAddend1->lLength into x0

```
ldr x0, [sp, 56]
```

- Load oAddend1->aulDigits[3] into x0

```
ldr x0, [sp, 56]
```

```
mov x1, 3
```

```
ldr x0, [x0, x1, lsl 3]
```

- Store ulSum into oSum->aulDigits[5]

```
ldr x0, [sp, 24]
```

```
ldr x1, [sp, 40]
```

```
add x1, 8
```

```
mov x2, 5
```

```
ldr x3, [x1, x2, lsl 3]
```

```
str x0, [x3]
```

Solutions: bug hunting

- Load oAddend1->lLength into x0

```
ldr x0, [sp, 56] // put oAddend1 pointer into x0
ldr x0, [x0] // store the value of oAddend1->lLength into x0
```

- Load oAddend1->aulDigits[3] into x0

```
ldr x0, [sp, 56] // put oAddend1 pointer into x0
add x0, x0, 8 // add array offset
mov x1, 3 // put array index into x1
ldr x0, [x0, x1, lsl 3] // load oAddend1->aulDigits[3] into x0
```

- Store ulSum into oSum->aulDigits[5]

```
ldr x0, [sp, 24] // put ulSum into x0
ldr x1, [sp, 40] // put oSum pointer into x1
add x1, x1, 8 // add array offset correctly
mov x2, 5 // put array index into x2
str x0, [x1, x2, lsl 3] // store ulSum into oSum->aulDigits[5]
```

Overview

1 BigInt_add

2 Your turn: bug hunting

3 Summary

Summary

- BigInt_add: data structures, normal pattern, optimised pattern
- Using register addressing and scaled register offset addressing to access struct fields and array elements
- Dates to remember:
 - Assignment 5: due Tue, 4/21 at 9pm
 - Assignment 5 Quiz: Wed, 4/22 in lecture
- Acknowledgements
 - Most of the code in these slides is adapted from the precept materials, written by Prof. Bob Dondero
 - Can be accessed at <https://www.cs.princeton.edu/courses/archive/spring26/cos217/precepts/w12p1/>